Unit 4 Progress Check: FRQ Part C

1. The Passenger class is used to store information about an airplane passenger. A partial declaration of the Passenger class is shown.

```
public class Passenger
{
  /**
   *  Returns the loyalty rating of the passenger,
   *  guaranteed to be positive
   */
  public int getRating()
  { /* implementation not shown */ }

  /**
   *  Returns the passenger's unique ID
   */
  public String getID()
  { /* implementation now shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The FlightSeatingChart class represents the assignment of passengers to seats on an airplane flight. The arrangement of seats on an airplane is stored as a two-dimensional array, where each element may or may not be filled by a passenger. A partial declaration of the FlightSeatingChart class is shown.

```
public class FlightSeatingChart
{
  /** 2D array containing all passengers on a flight */
  private Passenger[][] chart;

  /**
   *  Moves the passenger with the highest loyalty rating seated in
   *  a row greater than upgradeRow to chart[upgradeRow][upgradeCol]
   *  Preconditions: upgradeRow and upgradeCol are valid row and
   *                 column indices, respectively, in chart.
   *                 chart[upgradeRow][upgradeCol] is null.
   *                 There is at least one passenger in a row
   *                 greater than upgradeRow.
   */
  public void upgrade(int upgradeRow, int upgradeCol)

  /* There may be instance variables, constructors,
     and methods that are not shown.  */
}
```

Unit 4 Progress Check: FRQ Part C

When an element of the two-dimensional array chart is accessed, the first index is used to specify the row and the second index is used to specify the column.

Write the FlightSeatingChart method upgrade. The method should find the passenger with the greatest loyalty rating seated in a row greater than upgradeRow and move that passenger to the seat at row upgradeRow and column upgradeCol. The chart element previously occupied by that passenger should be set to null. If there are multiple passengers that meet these criteria, then any of these passengers can be moved.

For example, suppose chart has the following contents. Each empty cell represents null. Each non-empty cell represents a passenger object, where the int is the passenger rating and the String is the passenger ID.

	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"		613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"		607 "mo002"	202 "ic676"
4			177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"	354 "cr321"	98 "pt108"		213 "rf999"

The call upgrade(1, 3) will move the passenger at row 3 column 3 to row 1 column 3, and will set row 3 column 3 to null. The contents of chart after the call are shown in the following figure. The shaded elements are the two that have changed.

Unit 4 Progress Check: FRQ Part C

	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"	607 "mo002"	613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"			202 "ic676"
4			177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"	354 "cr321"	98 "pt108"		213 "rf999"

The call upgrade(4, 0) will move the passenger at row 5 column 1 to row 4 column 0, and will set row 5 column 1 to null. The contents of chart after the call are shown in the following figure. The shaded elements are the two that have changed.

	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"	607 "mo002"	613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"			202 "ic676"
4	354 "cr321"		177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"		98 "pt108"		213 "rf999"

Complete method `upgrade`.

Unit 4 Progress Check: FRQ Part C

```
/**
 * Moves the passenger with the highest loyalty rating seated in
 * a row greater than upgradeRow to chart[upgradeRow][upgradeCol].
 * Preconditions: upgradeRow and upgradeCol are valid row and
 *                column indices, respectively, in chart.
 *                chart[upgradeRow][upgradeCol] is null.
 *                There is at least one passenger in a row
 *                greater than upgradeRow.
 */
public void upgrade(int upgradeRow, int upgradeCol)
```

Point 1

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all necessary elements in `chart` (no bounds errors)

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
               swapCol = c;
            }
         }
      }
   }
   chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
```

Unit 4 Progress Check: FRQ Part C

```
   chart[swapRow][swapCol] = null;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all elements

Responses will not earn the point if they

- use an indexed loop with correct bounds but fail to access elements inside the loop
- access the elements of the array incorrectly

Point 2

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getRating` on an element from the array

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
```

Unit 4 Progress Check: FRQ Part C

```
                swapCol = c;
             }
          }
       }
    }
    chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
    chart[swapRow][swapCol] = null;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access an element of `chart` incorrectly

Responses will not earn the point if they

- make any incorrect call to `getRating`

Point 3

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Determines whether a passenger rating is larger than some other value

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
```

Unit 4 Progress Check: FRQ Part C

```
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
               swapCol = c;
            }
         }
      }
   }
   chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
   chart[swapRow][swapCol] = null;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- identify elements incorrectly

Responses will not earn the point if they

- fail to call `getRating`

Point 4

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- compates an element of `chart` with `null`

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
```

Unit 4 Progress Check: FRQ Part C

```
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
               swapCol = c;
            }
         }
      }
   }
   chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
   chart[swapRow][swapCol] = null;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access an element of `chart` incorrectly

Responses will not earn the point if they

- use an incorrect comparison (e.g., `.equals`)

Point 5

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Identifies the eligible passenger with the greatest loyalty rating (algorithm)

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
```

Unit 4 Progress Check: FRQ Part C

```
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
               swapCol = c;
            }
         }
      }
   }
   chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
   chart[swapRow][swapCol] = null;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- confuse rows and columns

Point 6

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Moves the identified passenger to the specified location and sets the vacated position to `null` (algorithm)

Canonical Solution:

```
public void upgrade(int upgradeRow, int upgradeCol)
{
   int maxRating = 0;
   int swapRow = -1;
```

Unit 4 Progress Check: FRQ Part C

```
   int swapCol = -1;
   for (int r = upgradeRow + 1; r < chart.length; r++)
   {
      for (int c = 0; c < chart[0].length; c++)
      {
         if (chart[r][c] != null)
         {
            int curRating = chart[r][c].getRating();
            if (curRating > maxRating)
            {
               maxRating = curRating;
               swapRow = r;
               swapCol = c;
            }
         }
      }
   }
   chart[upgradeRow][upgradeCol] = chart[swapRow][swapCol];
   chart[swapRow][swapCol] = null;
}
```



0	1

The response earns the point.

Responses can still earn the point even if they

- fail to maintain a maximum rating

Responses will not earn the point if they

- return or print a value

Unit 4 Progress Check: FRQ Part C

2. The GameGrid class is used to move a game piece diagonally on a two-dimensional grid. A partial declaration of the class is shown.

```
public class GameGrid
{
  /**
   * A grid that a game piece moves on. Each element indicates the
   * direction the game piece should move next (either "NE", "NW",
   * "SE", or "SW").
   */
   private String[][] grid;

  /**
   * Computes and returns the number of moves that are taken
   * by the game piece when following a path through the grid
   * Preconditions: startRow is a valid row index.
   *                startCol is a valid column index.
   *                There exists a path that moves the game piece
   *                from the starting position to past the
   *                boundaries of the grid.
   * Postcondition: grid is unchanged.
   */
  public int navigate(int startRow, int startCol)
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

When an element of the two-dimensional array grid is accessed, the first index is used to specify the row and the second index is used to specify the column.

The navigate method has two parameters that represent the starting row and column of the game piece. Each element of the two-dimensional array grid is equal to one of four String values: "NE", "NW", "SE", or "SW". These values indicate the direction the game piece should be moved, as described in the following table.

Direction	Effect
"NE"	Decreases the current row by one and increases the current column by one

Unit 4 Progress Check: FRQ Part C

Direction	Effect
"NW"	Decreases the current row by one and decreases the current column by one
"SE"	Increases the current row by one and increases the current column by one
"SW"	Increases the current row by one and decreases the current column by one

The game piece continues to move according to the values in grid until the current row or current column is outside the boundaries of grid. The method should return the number of valid grid elements accessed.

For example, suppose grid has the following contents.

	0	1	2	3	4	5	6
0	"SE"	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	"SE"	"NE"	"SE"	"NE"	"SE"	"NE"
2	"NW"	"NW"	"NE"	"NW"	"SE"	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	"SW"	"NW"
4	"SW"	"NE"	"NW"	"NE"	"SE"	"NE"	"SW"

The call navigate(0, 0) will cause the game piece to move along the path indicated by the shaded elements in the following figure. The method should return 7 because there are seven elements accessed.

Unit 4 Progress Check: FRQ Part C

	0	1	2	3	4	5	6
0	"SE"	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	"SE"	"NE"	"SE"	"NE"	"SE"	"NE"
2	"NW"	"NW"	"NE"	"NW"	"SE"	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	"SW"	"NW"
4	"SW"	"NE"	"NW"	"NE"	"SE"	"NE"	"SW"

The call navigate(2, 1) will cause the game piece to move along the path indicated by the shaded elements in the following figure. The method should return 5 because there are five elements accessed.

	0	1	2	3	4	5	6
0	"SE"	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	"SE"	"NE"	"SE"	"NE"	"SE"	"NE"
2	"NW"	"NW"	"NE"	"NW"	"SE"	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	"SW"	"NW"
4	"SW"	"NE"	"NW"	"NE"	"SE"	"NE"	"SW"

Complete method navigate.

```
/**
 * Computes and returns the number of moves that are taken
 * by the game piece when following a path through the grid
 * Preconditions: startRow is a valid row index.
 *                startCol is a valid column index.
 *                There exists a path that moves the game piece
 *                from the starting position to past the
 *                boundaries of the grid.
 * Postcondition: grid is unchanged.
```

Unit 4 Progress Check: FRQ Part C

```
     */
    public int navigate(int startRow, int startCol)
```

Point 1

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all necessary elements in `grid` (no bounds errors)

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
           && (row >= 0) && (col >= 0))
   {
      count++;
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
      {
         row--;
         col--;
      }
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
```

Unit 4 Progress Check: FRQ Part C

```
   return count;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all necessary elements

Responses will not earn the point if they

- access the elements of the array incorrectly

Point 2

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls a `String` method to determine the contents of an element in the array

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
          && (row >= 0) && (col >= 0))
   {
      count++;
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
```

Unit 4 Progress Check: FRQ Part C

```
      {
         row--;
         col--;
      }
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
   return count;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access an element of `grid` incorrectly

Responses will not earn the point if they

- make any incorrect call to a `String` method

Point 3

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Identifies four cases based on comparison(s) with a value obtained from an array element

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
```

Unit 4 Progress Check: FRQ Part C

```
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
           && (row >= 0) && (col >= 0))
   {
      count++;
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
      {
         row--;
         col--;
      }
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
   return count;
}
```

0	✓ 1

The response earns the point.

Responses can still earn the point even if they

- access elements incorrectly

Responses will not earn the point if they

Unit 4 Progress Check: FRQ Part C

- use incorrect syntax for the comparison

Point 4

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Increments or decrements both the current row and current column being used by the traversal

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
           && (row >= 0) && (col >= 0))
   {
      count++;
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
      {
         row--;
         col--;
      }
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
```

Unit 4 Progress Check: FRQ Part C

```
   return count;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access elements incorrectly
- have a bounds error

Point 5

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Moves through the grid as directed by the contents of each grid element (algorithm)

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
           && (row >= 0) && (col >= 0))
   {
      count++;
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
      {
         row--;
         col--;
      }
```

Unit 4 Progress Check: FRQ Part C

```
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
   return count;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- confuse rows and columns
- have a bounds error

Point 6

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Returns the number of valid grid elements visited during the traversal (algorithm)

Canonical Solution:

```
public int navigate (int startRow, int startCol)
{
   int count = 0;
   int row = startRow;
   int col = startCol;

   while ((row < grid.length) && (col < grid[0].length)
           && (row >= 0) && (col >= 0))
   {
      count++;
```

Unit 4 Progress Check: FRQ Part C

```
      String dir = grid[row][col];
      if (dir.equals("NE"))
      {
         row--;
         col++;
      }
      else if (dir.equals("NW"))
      {
         row--;
         col--;
      }
      else if (dir.equals("SE"))
      {
         row++;
         col++;
      }
      else if (dir.equals("SW"))
      {
         row++;
         col--;
      }
   }
   return count;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- have a bounds error

Responses will not earn the point if they

- print the count instead of or in addition to returning it